1. The string sE labeling an edge perfectly matches a substring of s the input the string; this means s starts with sE, so we can break it up as s=sE+s’. In this case, we can traverse the edge to the children, and recurse on the input string s’.
2. There is an edge starting with the first character in s, but sE is not a prefix of s; they, however, must have a common prefix sP (at least one-character long); the action here depends on the operation we are running: for search, it’s a failure because it means there isn’t a path to the searched key; for insert, as we’ll see, it means we have to decompress that edge, breaking down sE.
3. The input string s is a prefix of the edge’s label sE: this is a special case of point #2, and can be handled similarly.
4. Finally, if we don't find a match for the first character, then we are sure we can’t traverse the trie any longer.

Now that we have clarified the high-level structure of radix trie’s nodes, let's delve into the algorithms: keeping in mind the considerations above, their behavior will flow naturally from trie’s methods.

Listing 6.12 class RadixTrie

```
class RTEdge
  #type RTNode
  destination

  #type string
  label
```